

Grid Class Documentation

Source File: 'Grid.h'
Namespace: dsp
Class Header: class Grid : public Object

Overview

The *Grid* class builds a grid of space and walls with dedicated start and exit.

Constructors

- Grid() (default constructor)
 - **Purpose:** Constructs a 4 by 4 grid with no walls whose start is at the upper left-hand corner and exit is at the lower right-hand corner.
- Grid(size_t r, size_t c)
 - **Purpose:** Constructs a *r* by *c* grid with no walls whose start is at the upper left-hand corner and exit is at the lower right-hand corner.
 - **Parameter(s):**
 - *r*: Unsigned integer.
 - *c*: Unsigned integer.
 - **Exception(s):**
 - **invalid_argument:** Occurs if either *r* or *c* is less than 1 or greater than 20.
- Grid(const Grid& obj) (copy constructor)
 - **Purpose:** Makes a deep copy of *obj*.
 - **Parameter(s):**
 - *obj*: Constant *Grid* reference object.

Assignment Operators

- operator=(const Grid& rhs)
 - **Purpose:** Makes a deep copy of *rhs*.
 - **Parameter(s):**
 - *rhs*: Constant *Grid* reference object.
 - **Return:** *this.

Destructor

- ~Grid() [virtual]
 - **Purpose:** Does nothing

Methods

- rows() const
 - **Purpose:** Retrieves the number of rows in its boundary.
 - **Return:** An unsigned integer.
- columns() const
 - **Purpose:** Retrieves the number of columns in its boundary.
 - **Return:** An unsigned integer.
- start(const Point& obj)
 - **Purpose:** Changes its start position to *obj* only if *obj* is not the end position and has the same boundary of the grid.
 - **Parameter(s):**
 - *obj*: Constant *Point* reference.
- end(const Point& obj)
 - **Purpose:** Changes its end position to *obj* only if *obj* is not the start position and has the same boundary of the grid.
 - **Parameter(s):**
 - *obj*: Constant *Point* reference.

- `start()` `const`
 - **Purpose:** Retrieves the start position.
 - **Return:** *Point* object.
- `end()` `const`
 - **Purpose:** Retrieves the end position.
 - **Return:** *Point* object.
- `mark(const Point& obj)`
 - **Purpose:** Marks *obj* position of the grid if *obj* is not the start, end, or wall position and boundary is the same as the grid.
 - **Parameter(s):**
 - *obj*: Constant *Point* reference.
- `available(const Point& obj) const`
 - **Purpose:** Returns true if *obj* is an open space or the end position; otherwise, returns false.
 - **Parameter(s):**
 - *obj*: Constant *Point* reference.
 - **Return:** A Boolean.
- `reset()`
 - **Purpose:** Clears all marked positions.
- `load(const string& obj)`
 - **Purpose:** Converts a binary string into the grid and returns true if the length of the binary string is equal to the length of the grid; otherwise, it returns false. The binary string is row-oriented, where 0 represents a space and 1 represents a wall.
 - **Parameter(s):**
 - *obj*: Constant string reference.
- `path(const string& obj) const`
 - **Purpose:** Returns true if *obj* represents a successful path from the start position to the end position; otherwise, it returns false. A path is a string of the letters 'L', 'R', 'U', and 'D' where the letters represent moving left, right, up, and down, respectively.
 - **Parameter(s):**
 - *obj*: Constant string reference.
 - **Return:** A Boolean.
- `toString() const` [overridden]
 - **Purpose:** Returns a string of the grid.
 - **Return:** A string.

Mon-Member Function

- `operator>>(istream& ii, Grid& obj)`
 - **Purpose:** Reads a binary string and invokes `load()`.
 - **Parameter(s):**
 - *ii*: *istream* reference object.
 - *obj*: *Grid* reference object.
 - **Return:** An *istream*.